

www.devexpress.com

EXPRESSMemDATA

by Developer Express

Developer
EXPRESS

Copyright © 2001 Developer Express Inc.

12312319901283
783461827468271
1234123478618273
12471264812648126
1418249082948912849
12461287468127648126
1246847621387462374678
`12687`637836871`3676723
123728738172381987987423984
2354237587385783475872385738257
1482948928492819482194891284921849218

Table of Contents

TdxMemData - Getting Started	3
TdxMemData	7
TdxMemData properties	8
TdxMemData methods	13
TdxMemData events	20
TdxMemField	21
TdxMemField properties	22
TdxMemField methods	24
TdxMemFields	25
TdxMemFields properties	26
TdxMemFields methods	28
TdxMemData.MoveCurRecordTo example	31
TdxMemData.SaveToStream, TdxMemData.LoadFromStream example	32
TdxMemData.DelimiterChar, TdxMemData.SaveToTextFile, TdxMemData.LoadFromTextFile example	33
Direct Access example	34
TdxMemIndex	35
TdxMemIndex properties	36
TdxMemIndex methods	37
TdxMemIndexes	38
TdxMemIndexes methods	39
How to Populate a MemData Table from Another MemData or Any Other TDataSet Descendant	40
How to Add Different Types of Data to MemData	41
How to Populate MemData via Direct Memory Access	44
Index	45

TdxMemData - Getting Started



ExpressMemData is an ultra-fast in-memory TDataSet descendant you can use to bind data from any datasource (including text, binary, ini and many other file types) in conjunction with all existing VCL data-aware controls.

ExpressMemData also supports the following:

- Sorting
- Grouping (via the ExpressQuantumGrid)
- Save or load to and from binary/text files
- Multi field Locate
- Filters

With MemData, you can create all standard field types and yes, BLOB fields are supported. You can populate MemData in two ways: As a TDataSet, by assigning values to its fields - which is easier and will be fully compatible when you decide to switch your MemData to a database back-end; Or by accessing the internal buffers of MemData as an array.

Possible uses for MemData include:

- As a fast way to store and display data in data-aware controls. MemData also offers an easy upgrade path should you decide to move to a multi-user database at a later time.
- As a temporary table, when processing complex reports (it is easier to populate MemData with your calculated records rather than designing complex reports).
- As a temporary table, when doing complex editing operations - you make all the edit operations with MemData, and apply all the changes to the database at once.
- As a local storage of remote data - you can allow your users to download tables to their local hard drive and then use MemData to browse the records, without the need for a live connection to the database.

When you use MemData in your application, no additional DLLs are required - all the functionality is built right into your program. MemData is ideal for fast, single-user applications where you require the ability to use existing data-aware control and maintain the functionality you have come to expect such as data validation.

Assume we need to create an application. In one of its modules a user can edit (edit, delete, add) the names of Internet friends.

Data Specifications:

Object countries (Id, country name) and object Internet friends (Id, first name, last name, nickname, e-mail, birthday, interests and country).

We could use standard BDE tables, but for the sake of this example, assume we don't want to use any database files, and of course, we don't want to distribute our application with powerful, but very large DLLs.

In this instance, we have two options:

Use only non-data aware controls.

Use TdxMemData.

The first solution is a good one if you have lots of time or are not concerned with data validation, such as required values or correct values. Obviously, you have to do an incredible amount of work to implement controls such as lookups, etc.

Let's now show you how easy it is to use our TdxMemData to resolve just this type of a problem.

1. Drop two TdxMemData components onto a form. Name the first mdCountries, the second mdIntFriends.
2. Double click on the mdCountries component to call the MemData Field Editor screen and press the add button. The New Field form should now be active:

To add a new field you must fill:

Field Name

Type

Size (for string fields only)

Field Type (Default data)

Lookup definition for lookup fields only

New Field

Field Properties

Name: Component:

Type: Size:

Field Type

☒ Data ☐ Calculated ☐ Lookup

Lookup Definition

Key Field: Dataset:

Lookup Field: Result Field:

OK Cancel

3. Create two field ID (Integer) and Name (String, char(50)).
4. Double click on the mdIntFriends component and add its fields:

ID (Integer), FirstName (String char(25)), LastName (String char(25)), NickName (String char(10)), Email (String char(80)), birthday ((Date), interests (String char(250)) and CountryID (Integer).

5. Now we have to add the lookup field to bind `mdIntFreinds.CountryID` <R> `mdCountries.ID`

Name it `CountryName`, type `ftString`, Size 50, field type – lookup and lookup definition:

Key Field – `CountryID`, DataSet - `mdCountries`, Lookup Field - `ID` and Result Field – `Name`.

Now we have a lookup field. In both the Borland DBGrid and ExpressGrid™ by Developer Express Inc., this will be represented as a lookup combo (in the ExpressGrid, it is represented as an incremental lookup combo).

6. Now we have to set correct ID field values for our `TdxMemData` components. Write the following event handler for `onAfterInsert`:

```
procedure TForm1.dmCountriesAfterInsert(DataSet: TDataSet);
begin
    DataSet.FindField('ID').AsInteger :=
        DataSet.FindField('recid').AsInteger;
end;
```

You can use this code for both components: `dmCountries` and `dmIntFriends`. 'RECID' is the field that is created for each `TdxMemData`. You can see it at design time in the Object Inspector or `TdxMemData` fields editor.

It is a unique, Auto Incremental field. In this demo, we use it as an index field to get the next value for our ID fields. It can be used as a `KeyField` in the `dxDBGrid` control.

7. Now we have to write code to save and restore data in a text file. Here it is:

```
//Loads the data on the form OnCreate event, checks if the file exists
procedure TForm1.FormCreate(Sender: TObject);
var
    FileName : String;
Begin
    FileName := ExtractFileDir(Application.ExeName) + '\countries.txt';
    If FileExists(FileName) then
        mdCountries.LoadFromTextFile(FileName);
    FileName := ExtractFileDir(Application.ExeName) + '\intfriends.txt';
    If FileExists(FileName) then
        mdIntFriends.LoadFromTextFile(FileName);
end;

//Saves the data on the form destroy event
procedure TForm1.FormDestroy(Sender: TObject);
begin
    mdCountries.SaveToTextFile(ExtractFileDir(Application.ExeName) +
'\countries.txt');
mdIntFriends.SaveToTextFile(ExtractFileDir(Application.ExeName) +
'\intfriends.txt');
end;
```

8. Now drop two DataSources and name them dsCountries and dsIntFriends, then link them with your dxMemData components.

9. Set Active to True for both TdxMemData components. The final step is to link your DataSources with individual data controls.

And that's all you need to do!

TdxMemData

The TdxMemData is directly inherited from the TDataSet component.

Unit

dxMemData

```
TdxMemData = class(TDataSet);
```

Description

The TdxMemData works directly with memory and allows you to work with small amounts of data with lightening fast speeds.

The TdxMemData supports:

Calculated and lookup fields

Sorting by any field, including calculated and lookup fields

Bookmarks (multiple select for grid controls)

Locate, lookup methods

Note: This component does not work with Delphi™ 2 and CPP Builder™ 1.

Hierarchy

```
graph TD
    TdxMemData --> TDataSet
    TDataSet --> TComponent
    TComponent --> TPersistent
    TPersistent --> TObject
```

TdxMemData properties

In TdxMemData

Data	public / read only	ProgrammedFilter	public / read / write
DelimiterChar	public / read / write	RecIdField	public / read only
FilterList	public / read / write	ReadOnly	published / read / write
Indexes	published / read / write	SortedField	published / read / write
IsLoading	public / read / write	SortOptions	published / read / write

Derived from TDataset

Active	published / read / write
Filter	public / read / write

Data property

read only

Returns the MemData's fields definitions.

```
property Data : TdxMemFields;
```

Description

Use the Data property to get access to the MemData's fields definitions.

Example

Direct Access example

DelimiterChar property

read / write

Determines the separator char.

```
property DelimiterChar : Char;
```

Description

Use the DelimiterChar property to determine the column separator character used by MemData when storing or restoring data within it, using the SaveToTextFile and LoadFromTextFile methods.

Examples

DelimiterChar, SaveToTextFile, LoadFromTextFile examples

See also

TdxMemData.LoadFromTextFile; TdxMemData.SaveToTextFile

FilterList property**read / write**

Determines the list of filtered records.

```
property FilterList : TList;
```

Description

The FilterList property is used to maintain a list of filtered records. If the ProgrammedFilter property is set to True, then a developer is responsible for setting the records for this list; Otherwise it is generated automatically with the help of the FilterRecord event.

See Also

TdxMemData.ProgrammedFilter

Indexes property**read / write**

Determines indexes for a specific MemData control.

```
property Indexes: TdxMemIndexes;
```

Description

With ExpressMemData, a user can index data by one field similar to indexing using Paradox tables. The Index property is used to work with these indexes via a TdxMemIndexes object.

IsLoading property**read / write**

Indicates whether the data loading process is active.

```
property IsLoading : Boolean;
```

Description

Use this property to know whether the MemData is loading data. You can also employ this property when using your own methods of data loading.

ProgrammedFilter property**read / write**

Determines the manner in which the filtering feature is applied to the ExpressMemData.

property ProgrammedFilter : **Boolean**;

Description

The ProgrammedFilter property is used for faster filtering. If this property is set to True, then the OnFilterRecord event is disabled and a developer is responsible for setting the records for the list of filtered records via the FilterList property. If the ProgrammedFilter is set to False, then the list of filtered records is generated automatically with the help of the FilterRecord event.

See Also

TdxMemData.FilterList

ReadOnly property**read / write**

Specifies whether the MemData is read-only.

property ReadOnly : **Boolean**;

Description

Use the ReadOnly property to prevent users from updating, inserting, or deleting data within a MemData component. By default, the ReadOnly property is False.

RecIdField property**read only**

Returns a pointer to the 'recid' field of a MemData component.

```
property RecIdField : TField;
```

Description

Use the RecIdField property to return a pointer to the 'recid' field of the MemData. 'RecId' is invisible and is created automatically during the creation of a TdxMemData component. The RecIdField is a unique field.

Note: If you do not have a unique field and wish to use the grouping capabilities of the ExpressGrid™, use this field as a KeyField property of the dxDBGrid.

Example

```
// Set a unique field after the insert operation.  
procedure TForm1.dxDMemData1AfterInsert(DataSet: TDataSet);  
begin  
    DataSet.FindField('id').AsInteger := DataSet.FindField('recid').AsInteger;  
end;
```

SortedField property**read / write**

Determines the name of the field by which a MemData component is sorted.

```
property SortedField : String;
```

Description

Use the SortedColumn property to specify the field name by which the Memdata is sorted.

Note: By default, the MemData sorts this field data in ascending order. To change the sort order, apply the SortOptions property. Additionally, using the SortOptions property, you can set case insensitive sorting.

See Also

TdxMemData.SortOptions

SortOptions property**read / write**

Specifies the sort type for a MemData component.

```
type TdxSortOption = (soDesc, soCaseInsensitive);  
    TdxSortOptions = set of TdxSortOption;  
property SortOptions : TdxSortOptions;
```

Description

Use SortOptions to specify sort type.

The following values are available:

soDesc	If active, ExpressMemData sorts in descending order, otherwise it sorts in ascending order.
soCaseInsensitive	Determines whether sorting is case insensitive.

By default, all these options are inactive.

See Also

TdxMemData.SortedField

TdxMemData methods

In TdxMemData

CreateFieldsFromDataSet	public	LoadFromTextFile	public
CreateFieldsFromStream	public	MoveCurRecordTo	public
FillBookMarks	public	SaveToBinaryFile	public
GetRecNoByFieldValue	public	SaveToStream	public
GetValueCount	public	SaveToTextFile	public
LoadFromDataSet	public	SetFilteredRecNo	public
LoadFromBinaryFile	public	SupportedFieldType	public
LoadFromStream	public	UpdateFilters	public

CreateFieldsFromDataSet method

Automatically creates fields from a specified dataset.

```
procedure CreateFieldsFromDataSet(DataSet : TDataSet);
```

Description

Call the CreateFieldsFromDataSet method to retrieve all fields from the dataset specified by the DataSet parameter. This method checks whether a field type is supported by the ExpressMemData and then adds the supported fields.

See Also

TdxMemData.CreateFieldsFromStream

CreateFieldsFromStream method

Automatically creates fields from a specified stream.

```
procedure CreateFieldsFromStream(Stream : TStream);
```

Description

Call the CreateFieldsFromStream method to retrieve all fields from a stream specified by the Stream parameter. This method checks whether a field type is supported by the ExpressMemData and then adds supported fields.

Note: A stream must contain data in MemData internal format (For example, data written to a stream by another MemData using the SaveToStream method).

See Also

TdxMemData.CreateFieldsFromDataSet

FillBookMarks method

Fills MemData bookmarks.

```
procedure FillBookMarks;
```

Description

Call this method only when you insert data using direct access without standard DataSet methods like Append or Insert.

GetRecNoByFieldValue method

The GetRecNoByFieldValue method returns a record number by a specific field value.

```
function GetRecNoByFieldValue(Value : Variant; FieldName:String): Integer;  
virtual;
```

Description

Call the GetRecNoByFieldValue method to obtain a specific RecNo. Use the Value and FieldName parameters to specify the required RecNo. If an applicable record is not found, the GetRecNoByFieldValue returns -1. If there is more than one record with a specific value, the RecNo of the first record is returned.

GetValueCount method

Determines the number of records that contain a specified field value.

```
function GetValueCount(FieldName : String; Value : Variant) : Integer;
```

Description

Call the GetValueCount method to determine how many ExpressMemData records contain the value specified by the Value parameter in the field specified by the FieldName parameter.

LoadFromDataSet method

Loads data from a specified dataset.

```
procedure LoadFromDataSet(DataSet : TDataSet);
```

Description

Call this method to load data from a dataset specified by the DataSet parameter.

Note: When retrieving data from a dataset, make certain that this dataset contains the fields supported by ExpressMemData.

See Also

TdxMemData.LoadFromBinaryFile; TdxMemData.LoadFromStream; TdxMemData.LoadFromTextFile; TdxMemData.SaveToBinaryFile; TdxMemData.SaveToStream; TdxMemData.SaveToTextFile

LoadFromBinaryFile method

Loads data from a binary file.

```
procedure LoadFromBinaryFile(FileName : String);
```

Description

Use this method to load data from a binary file specified by the FileName parameter.

Note: A file specified by the FileName must contain data in MemData internal format (For example, data written to a file by another MemData using the SaveToBinaryFile method).

Example

```
//Load data from a binary file
procedure TForm1.Button1Click (Sender: TObject);
begin
  with dxMemData1 do
    begin DisableControls;
      //Define comma as column separator
      LoadFromBinaryFile(ExtractFileDir(Application.ExeName) + 'mdata.bin');
      EnableControls;
    end;
end;
```

See Also

TdxMemData.LoadFromDataSet; TdxMemData.LoadFromStream; TdxMemData.LoadFromTextFile; TdxMemData.SaveToBinaryFile; TdxMemData.SaveToStream; TdxMemData.SaveToTextFile

LoadFromStream method

Loads data from a stream.

```
procedure LoadFromStream(Stream : TStream);
```

Description

Call LoadFromStream to populate the MemData with data from a stream. The Stream parameter specifies the name of a stream from which to read data.

Note: A stream must contain data in MemData internal format (For example, data written to a stream by another MemData using the SaveToStream method).

Example

SaveToStream, LoadFromStream example

See Also

*TdxMemData.LoadFromBinaryFile; TdxMemData.LoadFromDataSet;
TdxMemData.LoadFromTextFile; TdxMemData.SaveToBinaryFile; TdxMemData.SaveToStream;
TdxMemData.SaveToTextFile*

LoadFromTextFile method

Populates data from a text file.

```
procedure LoadFromTextFile(FileName : String); dynamic;
```

Description

Use this method to load data from a text file specified by the FileName parameter. By default, when loading data from a text file 'TABs' will be used as column separators. If you have a text file with a comma as a separating character, you must set the DelimiterChar property first.

Note: When you use the LoadFromTextFile method, you are not guaranteed that all data will be loaded properly. If regional settings are different from original settings with which the file was created, loading errors may occur. If you want to be certain that your file will be read properly, you have to use the SaveToBinaryFile and LoadFromBinaryFile methods.

Example

DelimiterChar, SaveToTextFile, LoadFromTextFile example

See Also

*TdxMemData.LoadFromBinaryFile; TdxMemData.LoadFromDataSet; TdxMemData.LoadFromStream;
TdxMemData.SaveToBinaryFile; TdxMemData.SaveToStream; TdxMemData.SaveToTextFile*

MoveCurRecordTo method

Moves the current record to a new position within the MemData.

```
procedure MoveCurRecordTo(Index : Integer);
```

Description

Use this method if you wish to move the current active record to another position specified by the Index parameter.

Example

TdxMemData.MoveCurRecordTo example

SaveToBinaryFile method

Saves data to a binary file.

```
procedure SaveToBinaryFile(FileName : String);
```

Description

Use this method if you want to save data to a binary file, specified by the FileName parameter.

Example

```
//Saves data to a binary file  
procedure TForm1.Button1Click(Sender: TObject);  
begin  
    dxMemData1.SaveToBinaryFile(ExtractFileDir(Application.ExeName) +  
    'mdata.bin');  
end;
```

See Also

*TdxMemData.LoadFromBinaryFile; TdxMemData.LoadFromStream; TdxMemData.SaveToStream;
TdxMemData.LoadFromTextFile; TdxMemData.SaveToTextFile*

SaveToStream method

Saves data to a stream.

```
procedure SaveToStream(Stream : TStream);
```

Description

Call the SaveToStream method to write the MemData's data to a stream. The Stream parameter specifies the name of the existing stream.

Example

SaveToStream, LoadFromStream example

See Also

TdxMemData.LoadFromStream; TdxMemData.LoadFromBinaryFile; TdxMemData.SaveToBinaryFile; TdxMemData.LoadFromTextFile; TdxMemData.SaveToTextFile

SaveToTextFile method

Saves data to a text file.

```
Procedure SaveToTextFile(FileName : String); dynamic;
```

Description

Use this method if you wish to save data to a text file, specified by the FileName parameter. By default, when saving data to a text file, 'TAB's will be used as column separators. If you want to use a different column separator, you have to use the DelimiterChar property.

Note: When you use the SaveToTextFile method, you are not guaranteed that all data will be loaded properly at a future point in time. If regional settings differ from original settings, with which the file was created, loading errors may occur. To be certain that your file will be read properly regardless of regional settings, you have to use the SaveToBinaryFile and LoadFromBinaryFile methods.

Example

DelimiterChar, SaveToTextFile, LoadFromTextFile example

See Also

TdxMemData.LoadFromTextFile; TdxMemData.LoadFromBinaryFile; TdxMemData.SaveToBinaryFile; TdxMemData.LoadFromStream; TdxMemData.SaveToStream

SetFilteredRecNo method

Sets the active filtered record.

```
procedure SetFilteredRecNo(Value: Integer);
```

Description

Call the SetFilteredRecNo method to activate a record whose index, in the list of filtered records, is equal to the Value property.

SupportedFieldType method

Determines whether a specific FieldType is supported by the TdxMemData.

```
function SupportedFieldType(AType: TFieldType): Boolean; virtual;
```

Description

Use the SupportedFieldType to determinate whether a field type is supported by the MemData (As determined by the AType parameter). If a given field type is supported, the SupportedFieldType returns True; Otherwise - False.

Note: With the release of v1.4, BLOB fields are supported.

UpdateFilters method

Updates MemData filters.

```
procedure UpdateFilters; virtual;
```

Description

Call this method to update the MemData filters which can be defined in the OnFilterRecord event. Use this method when making changes to filter definitions.

TdxMemData events

Derived from TDataSet

AfterCancel	published	BeforeEdit	published
AfterClose	published	BeforeInsert	published
AfterDelete	published	BeforeOpen	published
AfterEdit	published	BeforePost	published
AfterInsert	published	BeforeScroll	published
AfterOpen	published	OnCalcFields	published
AfterPost	published	OnDeleteError	published
AfterScroll	published	OnEditError	published
BeforeCancel	published	OnFilterRecord	published
BeforeClose	published	OnPostError	published
BeforeDelete	published		

TdxMemField

The TdxMemField is the object used by a MemData component to store specific memory field definitions.

Unit

dxMemData

```
TdxMemField = class(TCollectionItem);
```

Description

The TdxMemField object allows you to get access to the memory field definition. Each of MemData's memory fields is a list of pointers to field values.

Hierarchy

```

TObject
  TPersistent
    TCollectionItem
      TdxMemField
```

See Also

TdxMemData; TdxMemFields

TdxMemField properties

In TdxMemField

DataSet	public / read only	Index	public / read / write
Field	public / read only	MemFields	protected / read only
HasValues	public / read / write	Values	public / read / write

DataSet property

read only

Specifies a corresponding dataset.

```
property DataSet : TdxMemData;
```

Description

Use this property to get access to a corresponding MemData component.

Field property

read only

Specifies a corresponding field.

```
property Field : TField;
```

Description

Use this property to get access to a corresponding field definition.

HasValues property

read / write

```
property HasValues[Index:Integer]: Char;
```

Description

This property is used internally by the SaveToStream and Locate methods. You might never need to use it directly.

Index property**read / write**

Specifies the field definition's index number in the Fields property of a dataset.

property Index: **Integer**;

Description

Use the Index property to locate a field within a dataset. Change the value of the Index property to change the location of a field within a dataset. Changing the Index value affects the order in which fields are displayed in data grids, but not their position in the physical database tables.

Note: The field order can also be changed by dragging and dropping fields in the Fields editor at design time.

See Also

TdxMemFields; TdxMemFields.Items

MemFields property**read only**

Determines the corresponding TdxMemFields object.

property MemFields : TdxMemFields;

Description

Use the MemFields property to get access to a TdxMemFields object, which contains the MemField.

Values property**read / write**

Determines the values of a MemField.

property Values[Index:**Integer**] : Pointer;

Description

Use the Values property to read and write a specific MemField value determined by the Index parameter.

Example

Direct Access example

See Also

TdxMemField.ValueList

TdxMemField methods

In TdxMemField

AddValue	public
InsertValue	public

AddValue method

Adds a new field definition to a TdxMemField object instance.

```
procedure AddValue(const Buffer : PChar);
```

Description

Call the AddValue method to add a new field definition to the end of the list of available field definitions for a specific field. The Buffer parameter determines the value to add. When added, the new value can be displayed.

See Also

TdxMemField.InsertValue

InsertValue method

Inserts a new field definition to a TdxMemField object instance.

```
procedure InsertValue(AIndex : Integer; const Buffer : PChar);
```

Description

Call the InsertValue method to insert a new field definition to the list of available field definitions of a specific field. The AIndex parameter determines the position at which a new value is inserted. The Buffer parameter determines the value to add. When added a new value can be displayed.

See Also

TdxMemField.AddValue

TdxMemFields

TdxMemFields is a collection of TdxMemField objects used by a MemData component to store the memory fields definitions.

Unit

dxMemData

```
TdxMemFields = class(TCollection);
```

Description

TdxMemFields is a container for MemData's memory fields. It is a collection of lists, where each item is a TdxMemField object.

Hierarchy

```
Object
  |
  +--TPersistent
      |
      +--TCollection
          |
          +--TdxMemFields
```

TdxMemFields properties

In TdxMemFields

Count	public / read only	Items	public / read only
DataSet	public / read only	RecordCount	public / read only

Count property

read only

Indicates the number of memory fields in a TdxMemFields collection.

property Count : **Integer**;

Description

Use the Count property to obtain the number of memory fields.

See Also

TdxMemFields.GetCount

DataSet property

read only

Specifies a corresponding dataset.

property DataSet : TdxMemData;

Description

Use this property to get access to a corresponding MemData component.

Items property

read only

Lists the MemField definitions that describe each memory field in a MemData component.

property Items[Index:**Integer**] : TdxMemField;

Description

To access a specific MemField's definition specify the Index parameter. The Index parameter is in the range from 0 to Count - 1.

Example

Direct Access example

RecordCount property**read only**

Indicates the total number of records in a corresponding MemData.

property RecordCount : **Integer**;

Description

The RecordCount property returns the total number of records in a corresponding MemData.

Example

Direct Access example

See Also

TdxMemFields.DeleteRecord; TdxMemFields.InsertRecord

TdxMemFields methods

In TdxMemFields

Add	protected	GetCount	public
AddField	protected	IndexOf	public
Clear	protected	InsertRecord	protected
DeleteRecord	protected	RemoveField	protected

Add method

Adds a new field to the memory fields collection.

```
function Add(AField : TField) : TdxMemField;
```

Description

Call the Add method to add a new memory field specified by the AField parameter to the end of the available fields list. This method returns the newly created TdxMemField object.

See Also

TdxMemFields.AddField; TdxMemFields.RemoveField

AddField method

Adds a new field to the memory fields collection.

```
procedure AddField(Field : TField);
```

Description

The AddField method is used internally to add a new memory field specified by the AField parameter. If this field is not in the list of available fields, it is added to the end of the list.

See Also

TdxMemFields.Add; TdxMemFields.RemoveField

Clear method

Deletes all memory fields from the TdxMemFields collection.

```
procedure Clear;
```

Description

The Clear method frees the memory used to store the memory fields list and all corresponding records.

See Also

TdxMemFields.RemoveField

DeleteRecord method

Deletes a specific record.

```
procedure DeleteRecord(AIndex : Integer);
```

Description

Call the DeleteRecord method to delete a record specified by the AIndex parameter. The first record has an index of 0, the second record has an index of 1, etc. The last record has an index of Count-1.

See Also

TdxMemFields.Count; TdxMemFields.InsertRecord; TdxMemFields.RecordCount

GetCount method

Returns the number of memory fields in a TdxMemFields collection.

```
function GetCount : Integer;
```

Description

The GetCount method is used internally to obtain the number of memory fields.

See Also

TdxMemFields.GetCount

IndexOf method

Locates the MemField object in a MemFields array.

Function IndexOf(Field : TField): TdxMemField;

Description

Call the IndexOf to get the MemField specified by the Field parameter. If the required field is not found, IndexOf returns nil.

InsertRecord method

Inserts a new record.

procedure InsertRecord(Buffer: Pointer; AIndex : **Integer**; Append: **Boolean**);

Description

Call the InsertRecord method to insert a new record into the ExpressMemData. The Buffer parameter contains a pointer to the inserted record. AIndex determines the index at which to insert this record. The first record has an index of 0, the second record has an index of 1 etc. The last record has an index of Count-1. The Append parameter determines whether to add or insert a new record. If this parameter is True, then a record is simply added to the end of all records; If False, it is inserted at the specified position and indexes of all subsequent records are recalculated.

See Also

TdxMemFields.DeleteRecord; TdxMemFields.RecordCount

RemoveField method

Removes a field from the memory fields collection.

procedure RemoveField(Field : TField);

Description

Call the RemoveField method to delete a field specified by the Field parameter from the list of available memory fields in the TdxMemFields collection.

See Also

TdxMemFields.Add; TdxMemFields.AddField

TdxMemData.MoveCurRecordTo example

This example demonstrates how to move a specific record.

```
//Moves a specified record to the beginning of the MemData
procedure TForm1.Button3Click(Sender: TObject);
begin
    dxMemData1.DisableControls;
    //Move record
    dxMemData1.MoveCurRecordTo(0);
    dxMemData1.EnableControls;
    //Go to the first record
    dxMemData1.First;
end;
```

TdxMemData.SaveToStream, TdxMemData.LoadFromStream example

This example demonstrates how to save and load a MemData instance using a stream.

```
//Copies data from one MemData component to another
procedure TForm1.Button1Click(Sender: TObject);
var
    fMem : TMemoryStream;
begin
    fMem := TMemoryStream.Create;
    //Saves the 1st MemData data to a stream
    dxMemData1.SaveToStream(fMem);
    //Sets the Stream position
    fMem.Position := 0;
    //Populates the 2nd MemData data from the stream, containing data from the
    1st MemData
    dxMemData2.LoadFromStream(fMem);
    fMem.Free;
end;
```

TdxMemData.DelimiterChar, TdxMemData.SaveToTextFile, TdxMemData.LoadFromTextFile example

This example demonstrates how to save and load a MemData instance using a text file.

```
//Saves data to a text file with commas as column separators

procedure TForm1.Button1Click(Sender: TObject);
begin
    //Defines commas as column separators
    dxMemData1.DelimiterChar := ',';
    dxMemData1.SaveToTextFile(ExtractFileDir(Application.ExeName) +
    'mdata.txt');
end;

//Loads data from a text file with commas as column separators
procedure TForm1.Button2Click (Sender: TObject);
begin
    with dxMemData1 do begin
        DisableControls;
        DelimiterChar := ',';
        //Define comma as column separator
        LoadFromTextFile(ExtractFileDir(Application.ExeName) + 'mdata.txt');
        EnableControls;
    end;
end;
```

Direct Access example

This examples demonstrates how to obtain access to each value of a MemData component.

```
procedure TForm1.Button1Click(Sender: TObject);  
var  
    i, j : Integer;  
    s : string;  
begin  
    with dxMemData do  
        for i := 0 to RecordCount - 1 do begin  
            s := '';  
            for j := 0 to Fields.Count - 1 do begin  
                if j = 0 then //determines the RecId field  
                    Continue;  
                if s <> '' then  
                    s := s + ' ' + dxMemData.Data.Items[j].Values[i]  
                else  
                    s := dxMemData.Data.Items[j].Values[i];  
            end;  
            Memo.Lines.Add(s);  
        end;  
    end;  
end;
```

TdxMemIndex

TdxMemIndex is the object used by a MemData component to store a specific index for a memory field.

Unit

dxMemData

```
TdxMemField = class(TCollectionItem);
```

Description

With ExpressMemData, a user can index data by one field similar to indexing applied to Paradox tables. The TdxMemIndex object allows you to work with a single index.

Hierarchy

TObject

TPersistent

TCollectionItem

TdxMemIndex

TdxMemIndex properties

FieldName published / read / write

FieldName property

read / write

Determines the field name to be indexed.

property FieldName: String;

Description

Use the FieldName property to specify the index for ExpressMemData. If this property contains a valid field name, then that index determines the sort order of records.

Note: *Only one field name can be specified in each index.*

TdxMemIndex methods

GetMemData	protected	Prepare	protected
GotoNearest	protected		

GetMemData method

Determines a MemData instance.

```
function GetMemData: TdxMemData;
```

Description

The GetMemData method returns a specific MemData instance to which the current index is applied.

GotoNearest method

Locates a record with a specific value in ExpressMemData.

```
function GotoNearest(const Buffer : PChar; var Index : Integer) : Boolean;
```

Description

The GotoNearest method is called to locate a record that contains the value specified by the Buffer parameter in the current MemData instance. The Index parameter determines the index of the resultant record. This method returns True if the search was successful; Otherwise False.

Prepare method

Prepares a MemData instance to use the current index.

```
procedure Prepare;
```

Description

Since ExpressMemData allows you to create several indexes, this method rebuilds a MemData instance to apply the new index.

TdxMemIndexes

TdxMemIndexes is a collection of TdxMemIndex objects used by a MemData component to maintain indexes for memory field definitions.

Unit

dxMemData

```
TdxMemIndexes = class(TCollection);
```

Description

With ExpressMemData, a user can index data by one field similar to indexing applied to Paradox tables. The TdxMemIndexes object maintains a list of these indexes. Each item of this list represents an independent index against which a sort order can be based.

Hierarchy

```
Object
  Persistent
    Collection
      TdxMemIndexes
```

TdxMemIndexes methods

Add	public
GetIndexByField	public

Add method

Adds a new index to the collection of available indexes.

```
function Add: TdxMemIndex;
```

Description

Call the Add method to create a new item in the collection of indexes in the ExpressMemData.

GetIndexByField method

Returns a specific TdxMemIndex instance.

```
function GetIndexByField(AField: TField): TdxMemIndex;
```

Description

Since each memory field can contain only a single index, the GetIndexByField method allows you to locate a specific index that corresponds to the field specified by the AField parameter.

How to Populate a MemData Table from Another MemData or Any Other TDataSet Descendant

This article demonstrates how to populate a MemData table, with another MemData or any other TDataSet descendant. To accomplish this, you should use the LoadFromDataSet method. Here is the code:

```
// Delphi code
dxMemData1.Close;
dxMemData1.LoadFromDataSet(dxMemData2);

// C++Builder code
dxMemData1->Close();
dxMemData1->LoadFromDataSet(dxMemData2);
```

How to Add Different Types of Data to MemData

This article illustrates the quickest way to populate MemData fields. You can perform this by directly accessing MemData data. For instance, to add new data to a field with #0, you should make use of the mData.Data.Items[0].AddValue string.

Additionally, to add different types of data, you should use the VariantToMemDataValue global procedure declared in the dxmdaset unit, which converts your data so that it's compatible with the MemData memory field data.

```
// Delphi code
procedure TForm1.Button1Click(Sender: TObject);
var
  I: Integer;
  P: Pointer;
begin
  // MemData must be active
  mData.Active := True;
  // Disable the data controls to prevent the flickers
  mData.DisableControls;
  GetMem(P, 255);
  try
    for I := 1 to 10 do
      begin
        mData.Data.Items[0].AddValue(@i);
        // string
        VariantToMemDataValue('Number ' + IntToStr(I), P, mDataString);
        mData.Data.Items[1].AddValue(P);
        // Integer
        mData.Data.Items[2].AddValue(@I);
        // Boolean
        VariantToMemDataValue((I mod 2) = 0, P, mDataBoolean);
        mData.Data.Items[3].AddValue(P);
        // Float
        VariantToMemDataValue(1 / I, P, mDataFloat);
        mData.Data.Items[4].AddValue(P);
        // Date
        VariantToMemDataValue(Now + I, P, mDataDate);
        mData.Data.Items[5].AddValue(P);
        // Date and time
        VariantToMemDataValue(Now + 1 / I, P, mDataDateTime);
        mData.Data.Items[6].AddValue(P);
```

```

        end;
        mData.FillBookmarks;
        mData.First;
    finally
        FreeMem(P);
        // Enabled the data controls
        mData.EnableControls;
    end;
end;

// C++Builder code
void __fastcall TForm1::Button1Click(TObject *Sender)
{
    // MemData must be active
    mData->Active = true;
    // Disable the data controls to prevent the flickers
    mData->DisableControls();

    char* P = new char [255];
    try
    {
        for (int i = 1; i <= 10; i++)
        {
            mData->Data->Items[0]->AddValue((char *)&i);
            // string
            VariantToMemDataValue("Number " + IntToStr(i), P, mDataString);
            mData->Data->Items[1]->AddValue(P);
            // Integer
            mData->Data->Items[2]->AddValue((char *)&i);
            // Boolean
            VariantToMemDataValue(i % 2 == 0, P, mDataBoolean);
            mData->Data->Items[3]->AddValue(P);
            // Float
            VariantToMemDataValue((float)1 / (float)i, P, mDataFloat);
            mData->Data->Items[4]->AddValue(P);
            // Date
            VariantToMemDataValue((int)Now() + i, P, mDataDate);
            mData->Data->Items[5]->AddValue(P);
            // Date and time
            VariantToMemDataValue((double)Now() + (float)1 / (float)i, P,
mDataDateTime);
            mData->Data->Items[6]->AddValue(P);

```

```
    }
    mData->FillBookMarks();

    mData->First();
}
__finally
{
    delete [] P;
    // Enabled the data controls
    mData->EnableControls();
}
}
//-----
```

How to Populate MemData via Direct Memory Access

This example demonstrates how to append a new record that contains two fields (an integer and a string) to a MemData component. Remember to call the FillBookmarks and Last methods.

```
// Delphi code
var
i: Integer;
i := 99999;
// Determines RecId field
dxMemData1.Data.Items[0].AddValue(@i);
// Determines integer field
dxMemData1.Data.Items[1].AddValue(@i);
// Determines string field
dxMemData1.Data.Items[2].AddValue(PChar('New value'));
dxMemData1.FillBookMarks;
dxMemData1.Last;

// C++Builder code
int i = 99999;
// Determines RecId field
dxMemData1->Data->Items[0]->AddValue((char *)&i);
// Determines integer field
dxMemData1->Data->Items[1]->AddValue((char *)&i);
// Determines string field
dxMemData1->Data->Items[2]->AddValue(PChar("New value"));
dxMemData1->FillBookMarks();
dxMemData1->Last();
```

Index

A

Add	27, 38
AddField	27
AddValue	23

C

Clear	28
Count	25
CreateFieldsFromDataSet	12
CreateFieldsFromStream	12

D

Data	8
DataSet	25
DataSet	21
DeleteRecord	28
DelimiterChar	8
DelimiterChar example	32
Direct Access example	33

E

example	30, 31, 32
---------------	------------

F

Field	21
FieldName	35
FillBookMarks	13
FilterList	9

G

GetCount	28
GetIndexByField	38
GetMemData	36
GetRecNoByFieldValue	13
Getting Started	3
GetValueCount	13
GotoNearest	36

H

HasValues	21
-----------------	----

I

Index	22
Indexes	9
IndexOf	29
InsertRecord	29
InsertValue	23
IsLoading	9
Items	25

L

LoadFromBinaryFile	14
LoadFromDataSet	14
LoadFromStream	15
example	31
LoadFromTextFile	15
example	32

M

MemFields	22
-----------------	----

MoveCurRecordTo	16
example	30

P

Prepare	36
ProgrammedFilter	9

R

ReadOnly	10
RecIdField	10
RecordCount	26
RemoveField	29

S

SaveToBinaryFile	16
SaveToStream	17
example	31
SaveToTextFile	17
example	32
SetFilteredRecNo	18
SortedField	11
SortOptions	11
SupportedFieldType	18

T

TdxmemData	12
CreateFieldsFromDataSet	12
CreateFieldsFromStream	12
TdxMemData	7, 9, 13, 18
Data	8
DelimiterChar	8
FillBookMarks	13
FilterList	9
GetRecNoByFieldValue	13
Getting Started	3
GetValueCount	13
Indexes	9
IsLoading	9
LoadFromBinaryFile	14
LoadFromDataSet	14
LoadFromStream	15
LoadFromTextFile	15
MoveCurRecordTo	16
ProgrammedFilter	9, 10
ReadOnly	10
RecIdField	10
SaveToBinaryFile	16
SaveToStream	17
SaveToTextFile	17
SetFilteredRecNo	18
SortedField	11
SortOptions	11
SupportedFieldType	18
UpdateFilters	18
TdxMemData.DelimiterChar example	32
TdxMemData.LoadFromStream example	31
TdxMemData.LoadFromTextFile example	32
TdxMemData.MoveCurRecordTo example	30
TdxMemData.SaveToStream example	31
TdxMemData.SaveToTextFile example	32
TdxMemField	20, 22, 23
AddValue	23
DataSet	21
Field	21
HasValues	21
Index	22
InsertValue	23

MemFields	22	FieldName	35
Values	22	GetMemData	36
TdxMemFields	24, 25, 27, 28, 29	GotoNearest	36
Add	27	Prepare	36
AddField	27	TdxMemIndexes	37, 38
Clear	28	Add	38
Count	25	GetIndexByField	38
Dataset	25		
DeleteRecord	28	U	
GetCount	28	UpdateFilters	18
IndexOf	29		
InsertRecord	29	V	
Items	25	Values	22
RecordCount	26		
RemoveField	29		
TdxMemIndex	34, 35, 36		